

FDU 并行计算 2. 并行算法设计

本文参考以下教材:

- 并行计算: 结构·算法·编程 (第 3 版, 陈国良) 第 5, 6, 7, 8, 9 章

欢迎批评指正!

2.1 基础知识

2.1.1 分类

并行算法是由多个可同时执行、相互协调的进程组成的算法体系, 可从多个维度进行分类:

- 按问题性质
 - **数值**并行算法: 基于代数运算, 如矩阵运算、线性方程组求解等.
 - **非数值**并行算法: 基于比较或符号处理, 如排序、搜索、匹配等.
- 按执行方式
 - **同步**并行算法: 各进程执行过程中需相互等待并进行同步.
 - **异步**并行算法: 各进程可独立执行, 无需全局同步.
- 按系统结构与通信方式
 - **共享存储**并行算法: 多个进程通过共享内存进行数据交换.
 - **分布存储**并行算法: 进程位于不同节点, 通过通信链路协同工作.
- 按算法确定性
 - **确定性**并行算法: 算法执行路径和结果完全由输入决定.
 - **随机**并行算法: 算法在执行过程中引入随机选择以决定后续操作.

2.1.2 复杂度

并行算法的复杂性分析通常采用渐近表示与多维性能指标相结合的方法.

- 渐近复杂度表示
 - **上界** $O(g(n))$: 描述算法复杂度的最大增长趋势.
 - **下界** $\Omega(g(n))$: 描述算法复杂度的最小增长趋势.
 - **紧致界** $\Theta(g(n))$: 同时给出上、下界, 刻画精确增长阶.
并行算法分析通常关注最坏情况复杂度, 以避免对输入分布作额外假设.
- 并行算法的主要复杂性指标
 - **运行时间** $t(n)$: 算法在给定并行模型上的执行时间, 通常包含计算时间和通信时间.
 - **处理器数** $p(n)$: 算法求解问题时所使用的处理器数量.
 - **成本** $c(n) = t(n)p(n)$: 反映时间与资源消耗的综合指标.
若 $c(n)$ 与最坏情况下的串行时间复杂度同阶, 则称算法为**成本最优**.
 - **总运算量** $W(n)$: 算法执行的总操作数, 与具体处理器数量无关.

Brent 定理:

设并行算法的

- 总工作量为 $W(n)$, 表示在理想的 PRAM 模型下执行的基本操作步数
- 关键路径长度 (或并行时间) 为 $T(n)$, 表示在无限处理器条件下的最短执行步数

则在使用 p 个处理器时, 其运行时间满足

$$t(n) = O\left(\frac{W(n)}{p} + T(n)\right).$$

这里:

- $W(n)/p$ 表示由于处理器数量有限引入的工作分摊代价.
- $T(n)$ 表示由算法固有的依赖关系决定的不可压缩时间下界.

当 $p = O(W(n)/T(n))$ 时, 算法成本 $c(n) = t(n)p = O(W(n) + pT(n))$ 与 $W(n)$ 渐近一致; 否则, 成本将大于 $W(n)$, 说明处理器未被充分利用.

2.1.3 同步 & 通信

同步是指在时间上强制多个并行进程在某些执行点相互等待, 以保证正确的执行顺序和对共享可写数据的互斥访问. 同步通常通过临界区与原子操作实现, 常见机制包括锁、屏障等.

(示例: 共享存储系统上的并行求和, 使用锁同步)

计算模型: MIMD 共享存储多处理器.

问题: 计算数组 $S = \sum_{i=0}^{n-1} a_i$.

设计思路:

- 每个处理器计算一部分局部和.
- 更新全局和时进入临界区, 防止写冲突.

并行划分:

- 处理器 P_i 负责下标 $i, i + p, i + 2p, \dots$ 的元素.
- 局部变量 L : 保存局部部分和.
- 全局变量 S : 保存最终结果.

通信是指在空间上对并发执行的进程进行数据交换, 是分布式并行算法的核心机制.

常见通信方式:

- **共享存储系统**: 通过全局读 / 写实现数据交换.
- **分布存储系统**: 通过消息传递进行通信.

(示例: 分布存储系统上的矩阵-向量乘法, 环形通信)

计算模型: MIMD 分布存储多计算机

问题: 计算 $y = Ax$

系统结构:

- p 个处理器构成逻辑环.

- 每个处理器有独立局部存储，仅能与左右邻居通信.

数据划分:

- 矩阵 A 按列划分为 $A_1, \dots, A_p$, 每块大小为 $n \times r$, 其中 $r = n/p$.
- 向量 x 划分为 $x_1, \dots, x_p$, 每块长度为 r .

基本思想:

- 每个处理器先计算局部乘积 $z_i = A_i x_i$.
- 再通过环形通信将部分结果累加.
- 最终由指定处理器得到完整结果 y .

消息传递显式表达了数据依赖关系.

2.1.4 计算模型

PRAM 模型 (Parallel Random Access Machine)

一种共享存储、同步并行的抽象模型，用于刻画并行算法的理想计算能力上限.

基本假设:

- 无限容量的共享存储器
- 多个功能相同的处理器
- 单位时间内完成: 算术运算、逻辑判断、一次存储访问
- 所有处理器锁步同步执行

读写冲突模型:

- **EREW**: 互斥读、互斥写 (最弱、最接近现实)
- **CREW**: 并发读、互斥写
- **CRCW**: 并发读、并发写 (最强)

优点:

- 表达简洁，便于算法设计与复杂度分析
- 屏蔽通信、同步、存储管理等细节
- 适合理论分析与模型间对比

缺点:

- 假设过于理想: 忽略通信延迟、带宽、访存竞争
- 同步锁步执行不符合真实并行机
- 不适合分布存储系统

异步 PRAM 模型 (APRAM)

在 PRAM 的基础上放弃全局时钟，引入异步执行 + 显式同步障.

主要特征:

- 每个处理器有局部存储、局部时钟、独立程序

- 无全局时钟，处理器异步执行
- 通过同步障协调阶段
- 全局存储仍然存在，但访问有约束

计算结构:

- 计算由多个 Phase (全局相) 构成
- 每个 Phase 中，处理器异步执行局部计算与读写，以同步障结束
- 同一 Phase 内不允许多个处理器访问同一存储单元

优点:

- 更接近真实并行机
- 保留 PRAM 的编程简洁性
- 能定量分析通信与同步代价

BSP 模型 (Bulk Synchronous Parallel)

通过粗粒度同步，在可编程性与性能建模之间取得平衡.

三大参数:

- p : 处理器数
- g : 通信带宽因子
- L : 全局同步周期

计算结构:

- 程序由一系列 Superstep 构成
- 每个 Superstep 包含局部计算、点到点通信、全局同步 (同步障)

优点:

- 明确区分计算与通信
- 屏蔽网络拓扑
- 易于设计可移植的并行算法
- 能有效模拟 PRAM

缺点:

- 全局同步代价高
- 消息必须跨 Superstep 使用
- 对不规则通信支持较弱

LogP 模型

刻画大规模分布式并行机 (MPC) 中通信的真实瓶颈，是通信主导型并行算法的首选模型.

四个参数:

- **L (Latency):** 通信延迟
- **o (Overhead):** 处理器发送/接收开销

- **g (Gap):** 连续通信的最小间隔 (带宽约束)
- **p (Processors):** 处理器数

关键假设:

- 点到点通信
- 网络容量有限
- 消息可能乱序到达
- 处理器既计算又通信

通信代价:

- 单条消息 $\approx 2o + L$
- 任意时刻在途消息数受 L/g 限制

优点:

- 准确反映通信延迟、带宽与处理器开销
- 不依赖具体网络拓扑
- 适合分析细粒度通信算法

缺点:

- 不显式支持广播、前缀、规约
- 不区分短消息与长消息

层次存储模型 (Hierarchical Memory Model)

并行性能瓶颈不仅在处理器, 还在存储层次与数据移动成本 (算得快 $\neq$ 取数据快).

传统并行计算模型 (如 PRAM、BSP、LogP) 通常假定单一主存、单位访存时间, 而现代计算机的存储系统具有明显的层次结构 (寄存器 $\rightarrow$ Cache $\rightarrow$ 主存 $\rightarrow$ 外存 / 远程存储), 不同层次访问代价差异巨大.

因此, 在分析算法性能时, **必须显式刻画数据所在层次及其移动代价**。

串行计算系统的层次存储模型:

- **HMM** (Hierarchical Memory Model) / HMM-BT (HMM with Block Transfer) 模型
 - 面向地址的层次存储模型
 - 访存代价是地址的单调函数, 适用顺序访问场景 (如磁带)
 - 局限性: 不适合随机访问、高速缓存体系
- **UMH** (Uniform Memory Hierarchy) 模型
 - 访存代价仅与存储层次编号有关
 - 每一层访问代价固定
 - 忽略数据复用, 假定不断从远层取数
 - 假定计算与存储访问可重叠, 强调预取
- **RAM(h)** 模型 (随机访问存储 (Random Access Memory, RAM) 模型推广至 h 层存储)
 - 明确区分寄存器、Cache、主存、外存和远程存储

- 用访问频率 $\times$ 层次代价衡量存储复杂度
- 不要求访存与计算重叠
- 不区分读写代价

并行计算系统的层次存储模型:

- **Memory-LogP** 模型 (LogP 模型的存储层次扩展)
 - 将通信延迟视为数据收集、存储层间复制、网络传输和中间件开销
 - 将通信与层次存储统一建模, 适合机群与 NUMA 系统
- **DRAM(h)** 模型 (分布式 RAM(h) 模型)

$$\text{DRAM}(h) = \text{RAM}(h) + \text{LogP}$$
 - 本地: RAM(h) 描述多级存储
 - 远程: 消息传递视为 "更远一层存储", 发送 = 写存储, 接收 = 读存储
 - 集合通信视为并发存储访问
- **HPM** (Hierarchical Parallelism and Memory) 模型
 - 同时刻画层次并行性和层次存储结构
 - 用三组函数描述系统:
 - 并行函数 (并行层次)、存储函数 (存储层次) 和绑定关系 (并行层 $\leftrightarrow$ 存储层)
 - 表达能力强, 但算法设计复杂

2.2 设计策略

2.2.1 快速排序

在为某一问题设计并行算法时, 一个自然且优先考虑的策略是:

从已有的串行算法出发, 通过分析其结构, 将其中可并行的部分并行执行.

由于长期积累了大量成熟而高效的串行算法, 直接并行化能够降低并行算法设计的难度, 继承原算法的正确性与数学性质 (如稳定性、收敛性等). 这在数值计算领域尤为常见.

在采用直接并行化策略时, 需要注意以下几点:

- **内在顺序性限制:**

若串行算法存在强顺序依赖 (每一步依赖前一步结果), 则难以直接并行化. 因此需要分析数据相关性, 消除不必要的顺序约束.
- **好串行算法 $\neq$ 好并行算法:**

某些优秀的串行算法, 并行化后未必能获得良好加速;
相反, 一些复杂度较高的串行算法 (如枚举算法) 在并行化后可能表现优秀.

快速排序 (quick sort) 算法的步骤如下:

- ① 计算数组长度, 若数组较小, 则使用插入排序; 否则执行 ②
- ② 选择 **pivot**, 并将数组分成小于 **pivot** 的部分与大于 **pivot** 的部分 (这一步可使用 Lomuto 划分算法或 Hoare 划分算法)
- ③ 递归处理左右子区间

平均情况 $\Theta(n \log n)$, 最坏情况 $\Theta(n^2)$.

划分步骤本身是顺序的, 两个递归子问题在逻辑上相互独立.

最自然的并行化策略是对左右子序列的递归调用并行执行.

这种方法不改变原算法结构且易于实现, 但并非成本最优.

因为如果划分步骤仍由单处理器完成, 其代价为 $\Theta(n)$, 即使使用 n 个处理器, 总成本仍为:

$$C(n) = p(n) \cdot t(n) = \Theta(n^2).$$

要获得成本最优的并行快速排序, 必须将划分过程也并行化.

核心思想是将快速排序视为构造一棵二叉划分树:

- 每个主元对应树中的一个结点
- 小于主元的元素进入左子树, 大于主元的进入右子树
- 多个主元的选择与比较可以并行完成

在 **PRAM-CRCW (并发读并发写) 模型**下, 可利用并发写冲突的解析规则, 实现高效并行选主元:

- 每个处理器保存一个元素
- 使用共享变量竞争写入, 自动 "筛选" 出主元
- 同时确定左右子树的代表处理器
- 并行构造整棵划分二叉树
- 最终通过中序遍历得到有序序列

Algorithm Parallel-Quicksort-BinaryTree (PRAM-CRCW)

Input : $A[1..n]$ // 待排序元素
 n processors // $P_1, P_2, \dots, P_n$

Output: A binary partition tree (LC, RC)

- $A[i]$: 处理器 P_i 保存的元素
- **root**: 共享变量, 记录当前子树主元处理器编号
- $f[i]$: 处理器 P_i 当前所属子树的主元处理器
- $LC[i]$: 主元 i 的左孩子 (处理器编号)
- $RC[i]$: 主元 i 的右孩子 (处理器编号)
- 约定 $n+1$ 表示 "空指针"

Begin

// ===== 初始化阶段 ($O(1)$) =====

parallel for each processor P_i do

$root = i$ // 并发写, CRCW 冲突自动选出一个主元

$f[i] = root$ // 记录当前主元

$LC[i] = n + 1$

$RC[i] = n + 1$

end parallel for

// ===== 逐层并行构造二叉树 =====

repeat

 parallel for each processor P_i where $i \neq f[i]$ do

 if ($A[i] < A[f[i]]$) OR

```

        ( A[i] = A[f[i]] AND i < f[i] ) then

        // 写入左子树
        LC[f[i]] = i           // 并发写：只保留一个代表
        if i = LC[f[i]] then
            exit                // Pi 成为左孩子
        else
            f[i] = LC[f[i]]     // 跟随新的子主元
        endif

    else
        // 写入右子树
        RC[f[i]] = i
        if i = RC[f[i]] then
            exit                // Pi 成为右孩子
        else
            f[i] = RC[f[i]]
        endif
    endif
end parallel for
until all processors exit

End

```

复杂度分析:

- 每一层树可在 $O(1)$ 时间内构造
- 树高为 $\Theta(\log n)$
- 总时间复杂度: $\Theta(\log n)$

2.2.2 串匹配

匹配的定义如下:

$$\text{match}(i) = \begin{cases} \text{true}, & \text{if } P = T[i : i + m - 1] \\ \text{false}, & \text{otherwise,} \end{cases}$$

其中 $T[1 : n]$ 为正文串, $P[1 : m]$ 为模式串, $m \leq n$.

朴素算法:

- 将 $T[1 : n]$ 分成 $n - m + 1$ 个长度为 m 的子串, 逐一与 P 比较
- 最坏情况的时间复杂度为 $O(mn)$

KMP 算法 (串行最优, [Demo](#)):

利用前缀信息, 通过失效函数

$$F(j) = \begin{cases} 0, & j = 1 \\ j - D(j), & j = 2, 3, \dots, m + 1 \end{cases}$$

决定模式串右移距离, 其中 $D(j)$ 为前缀 $P[1 : j - 1]$ 的最短周期.


```

Algorithm KMP_MATCH( $T[1..n]$ ,  $P[1..m]$ ,  $F[1..m+1]$ )
     $j = 1$  //  $j$ : 模式串  $P$  的指针, 表示当前已匹配到  $P[1..j-1]$ 
     $k = 1$  //  $k$ : 正文串  $T$  的指针, 表示当前正在比较的字符  $T[k]$ 

    // 保证  $P[j]$  与  $T[k]$  对齐后, 模式串不会越过正文串尾部,  $k - j$  表示当前模式左端在正文中的对齐起点
    while  $k - j \leq n - m$  do
        if  $T[k] = P[j]$  then
            // 当前字符匹配成功
             $k = k + 1$  // 正文指针右移, 继续比较下一个字符
             $j = j + 1$  // 模式指针右移, 扩展已匹配前缀长度

            if  $j = m + 1$  then
                // 模式串全部字符均已匹配成功
                match( $k - m$ ) = true // 在位置 ( $k - m$ ) 处找到一次完整匹配

                // 利用失效函数回退  $j$ , 以便寻找可能存在的重叠匹配
                 $j = F(j)$ 
            end if
        else
            // 当前字符匹配失败
             $j = F(j)$  // 根据失效函数, 将模式串向右滑动, 保留已匹配的最长前缀信息
            if  $j = 0$  then
                // 模式串已无可利用的前缀信息
                 $k = k + 1$  // 正文指针右移一位, 从下一个字符重新开始尝试匹配
                 $j = 1$  // 模式指针重置到起始位置
            end if
        end if
    end while
end while

```

时间复杂度为 $O(n)$.

但其指针的前进和回退具有强烈的顺序依赖, 无法直接并行化.

(失配见证函数)

对于位移 j ($1 < j \leq m/2$), 比较 $P[j:m]$ 与 $P[1:m-j+1]$.

若不相等, 则存在最小位置 w 使得 $P[w] \neq P[j-1+w]$.

记 $\text{WIT}(j) = w$, 否则定义 $\text{WIT}(j) = 0$.

直观理解:

- $\text{WIT}(j) = 0$: 说明长度 $m - j + 1$ 的前后子串完全相同 $\Rightarrow$ 存在周期 $j - 1$
- $\text{WIT}(j) = w$: 在位置 w 处出现失配

周期串与非周期串的判定:

- 非周期串: $\forall 2 \leq j \leq m/2, \text{WIT}(j) \neq 0$.
- 周期串: $\exists j \leq m/2, \text{WIT}(j) = 0$.

非周期性意味着任意两个不同对齐位置的匹配尝试, 不可能同时成功, 这使得**并行淘汰**成为可能.

给定两个候选起始位置 $p < q$, 通过如下过程定义**竞争函数** $\text{duel}(p, q)$.

令 $d = q - p$, 考虑:

- 若 $d > m/2$, 两个匹配窗口不重叠, 可独立验证

- 若 $d \leq m/2$, 查看 $\text{WIT}(d+1) = w$, 比较 $T[p+w-1]$ 和 $T[q+w-1]$.
若相等, 则它们要么同时匹配成功, 要么同时失配.
若不等, 则至多一个位置可能匹配成功.

(并行淘汰赛算法)

首先将候选位置 $C_0 = \{1, 2, \dots, n - m + 1\}$ 分成若干组 (如相邻两两一组), 并行执行决斗.

每组最多保留一个幸存者, 得到幸存者集合 C_1 , 其元素个数满足 $|C_1| \leq |C_0|/2$.

重复该过程 $O(\log n)$ 轮后, 仅剩 $O(1)$ 个候选位置.

最后对这些位置顺序验证模式串与正文串是否完全匹配.

```
Algorithm PARALLEL_ELIMINATION_MATCH( $T[1..n]$ ,  $P[1..m]$ , WIT)
    // 初始候选位置集合
     $C = \{1, 2, \dots, n - m + 1\}$ 

    while  $|C| > 1$  do
        // 将 C 按顺序分成若干组 (相邻两两一组)
        partition  $C$  into pairs  $(p_1, p_2), (p_3, p_4), \dots$ 

         $C_{\text{new}} = \emptyset$ 

        // 并行执行所有决斗
        parallel for each pair  $(p, q)$  do
            winner = DUEL( $p, q$ )
            if winner  $\neq 0$  then
                add winner to  $C_{\text{new}}$ 
            end if
        end parallel for

        // 若 |C| 为奇数, 最后一个元素直接进入下一轮
        if  $|C|$  is odd then
            add last element of  $C$  to  $C_{\text{new}}$ 
        end if

         $C = C_{\text{new}}$ 
    end while

    // 最终验证阶段
    for each position  $p$  in  $C$  do
        if  $T[p : p + m - 1] = P[1 : m]$  then
            match( $p$ ) = true
        end if
    end for
end Algorithm
```

2.2.3 最短路径

给定一个边权均为非负实数的有向无环图 $G = (V, E)$ (其中 $|V| = n$), 计算所有点对之间的最短路径.

定义**距离矩阵乘法** (又称 tropical / min-plus 乘法):

对于矩阵 $A, B \in (\mathbb{R} \cup \{\infty\})^{n \times n}$,

$$(C = A \otimes B), \quad C[i, j] = \min_k (A[i, k] + B[k, j]).$$

定义初始邻接矩阵 $D^{(1)}$:

$$D^{(1)}[i, j] = \begin{cases} 0, & i = j \\ w(i, j), & (i, j) \in E \\ \infty, & \text{otherwise} \end{cases}$$

定义最多使用 k 条边的最短距离矩阵 $D^{(k)}$.

递推关系 $D^{(2k)} = D^{(k)} \otimes D^{(k)}$.

由于 DAG 中最长路径长度 $\leq n - 1$, 只需计算:

$$D^{(1)}, D^{(2)}, D^{(4)}, \dots, D^{(2^{\lceil \log(n-1) \rceil})}.$$

```
Algorithm APSP_DAG_MATMUL(G = (V, E, w))
    n = |V|

    // Step 1: 初始化距离矩阵
    for i = 1 to n do
        for j = 1 to n do
            if i = j then
                D[i][j] = 0
            else if (i, j) ∈ E then
                D[i][j] = w(i, j)
            else
                D[i][j] = ∞
            end if
        end for
    end for

    // Step 2: 距离矩阵倍增
    k = 1
    while k < n do
        D = MIN_PLUS_MULTIPLY(D, D)
        k = 2 * k
    end while

    return D
end Algorithm
```

2.3 设计技术

2.3.1 划分

又是一场酣畅淋漓的赤史啊👁️